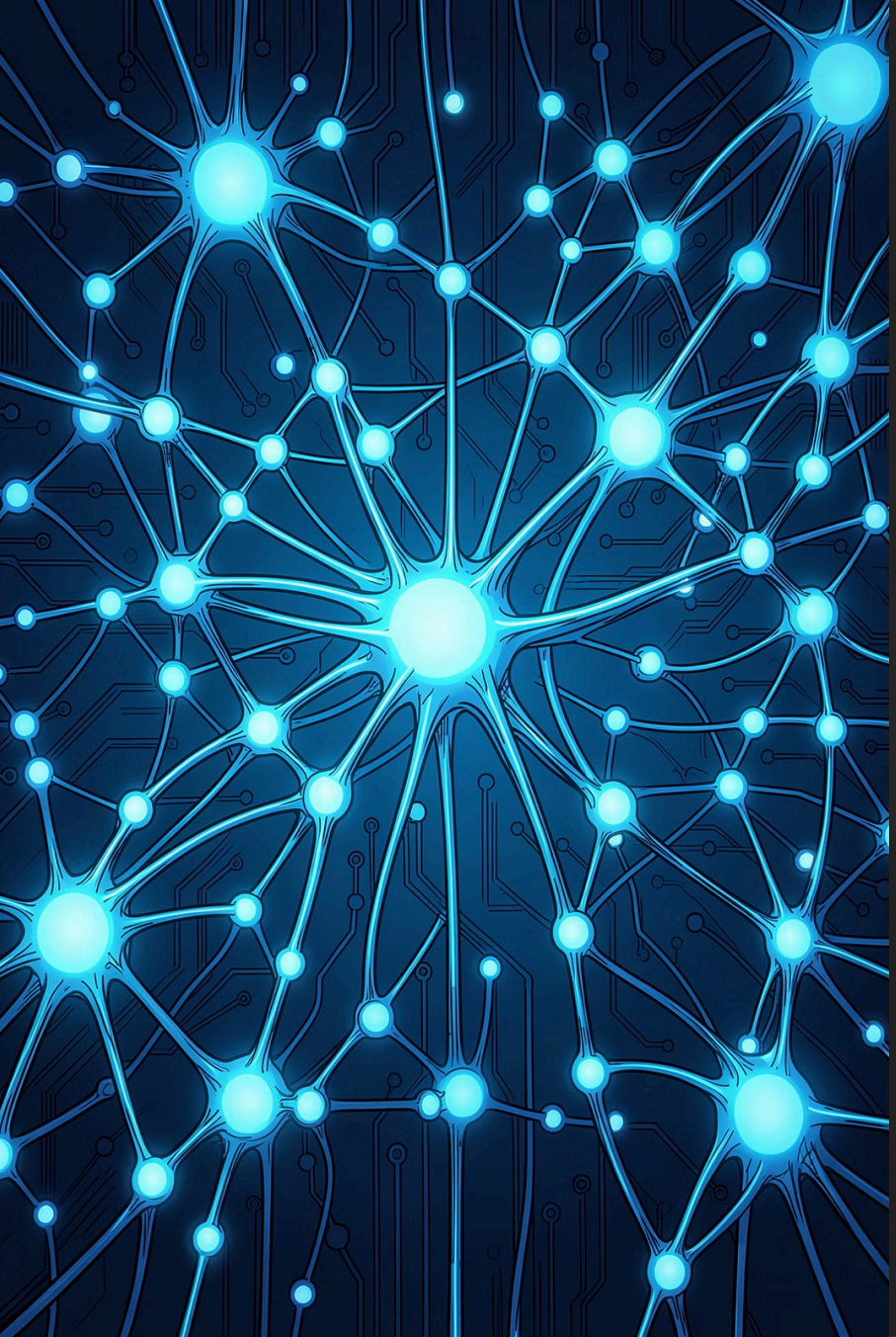


Chunking Strategies and the Effect of Chunk Size

Understanding RAG Systems

Narin Bilal

InnoVentures-Tech Summer Interǹship

(Retrieval-Augmented Generation)

The problem:

LLMs only know what they were trained on — they can't access private documents, recent news, or specialized knowledge on their own. This makes them prone to hallucination, confidently giving outdated or incorrect answers.

The solution:

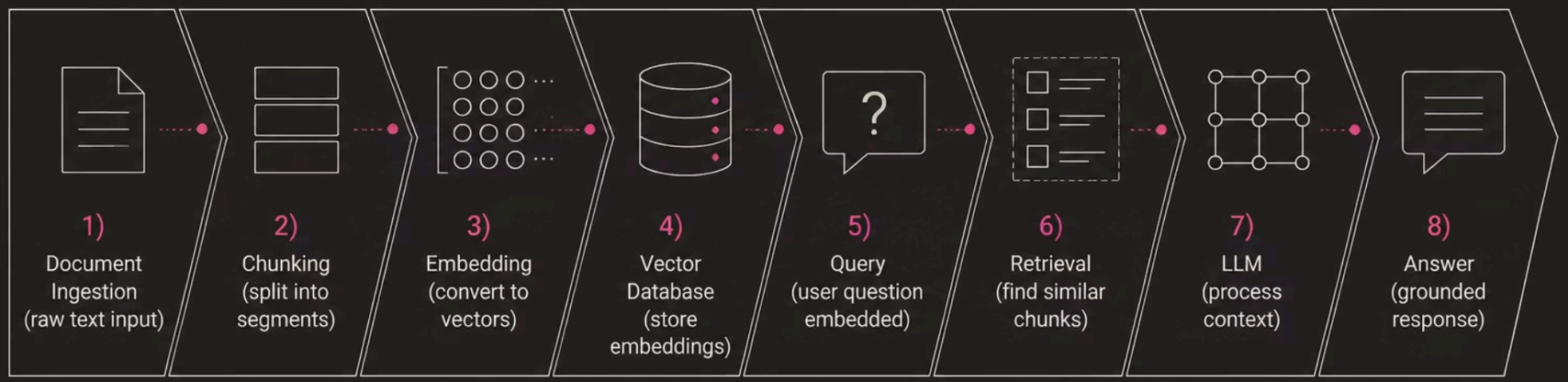
RAG combines a retrieval system (searches documents) with a language model (generates answers), so the model can look things up before answering — grounding its response in real information instead of guessing from memory alone.

Normal LLM (no RAG) = closed-book exam — answering purely from memory (training data), which may be outdated, incomplete, or **lead to hallucination** (confidently guessing wrong instead of admitting it doesn't know).

RAG = open-book exam — looking up the exact information before answering, so the response is accurate and up-to-date.

How RAG Works:

1. Documents are split into chunks
2. Chunks are converted into embeddings and stored in a vector database
3. User asks a question(convert the question to emmbedding vector)
4. System finds the most relevant chunks by comparing embeddings
5. Retrieved chunks + question are given to the LLM to generate an answer



What happens in the LLM steps :

1. **Prompt assembly** — The retrieved chunks and the user's question are combined into one prompt: "Answer the question using the information below" + chunks + question.
2. **Reading the context** — The LLM doesn't recall chunks from memory — it reads them in real time, like a passage just handed to it, and grounds its answer in that text.
3. **Answer generation** — If the chunks contain the answer, the model uses them. If not, a well-designed system responds "I don't have that information" instead of guessing.
4. **Source attribution** *(optional but common)* — Many RAG systems show which chunk/document the answer came from, so it's traceable.

The LLM doesn't remember — it reads the retrieved chunks and generates an answer grounded in them.

RETRIEVAL-AUGMENTED GENERATION

🔍 Retrieve

Search a vector database for relevant document chunks.

🧠 Generate

Feed retrieved context to the LLM for informed answers.

Why RAG Maters

Higher Accuracy

Answers are grounded in retrieved evidence, not model memory alone.

Always Up-to-Date

Update the vector store without retraining the entire model.

Reduced Hallucination

Retrieved context anchors responses to factual source material.

Private & Proprietary

Query internal documents without exposing them to the model's training.

What is Chunking?

Chunking is the process of splitting large documents into smaller, manageable segments before embedding. It is the critical preprocessing step that determines what context the LLM receives at retrieval time.

> The chunking strategy and chunk size directly control retrieval quality, context relevance, and in the end the accuracy of the final answer.

- Too small → missing context
- Too large → noise and diluted relevance
- Just right → precise, grounded answers

Chunking Strategies

Size-based chunking

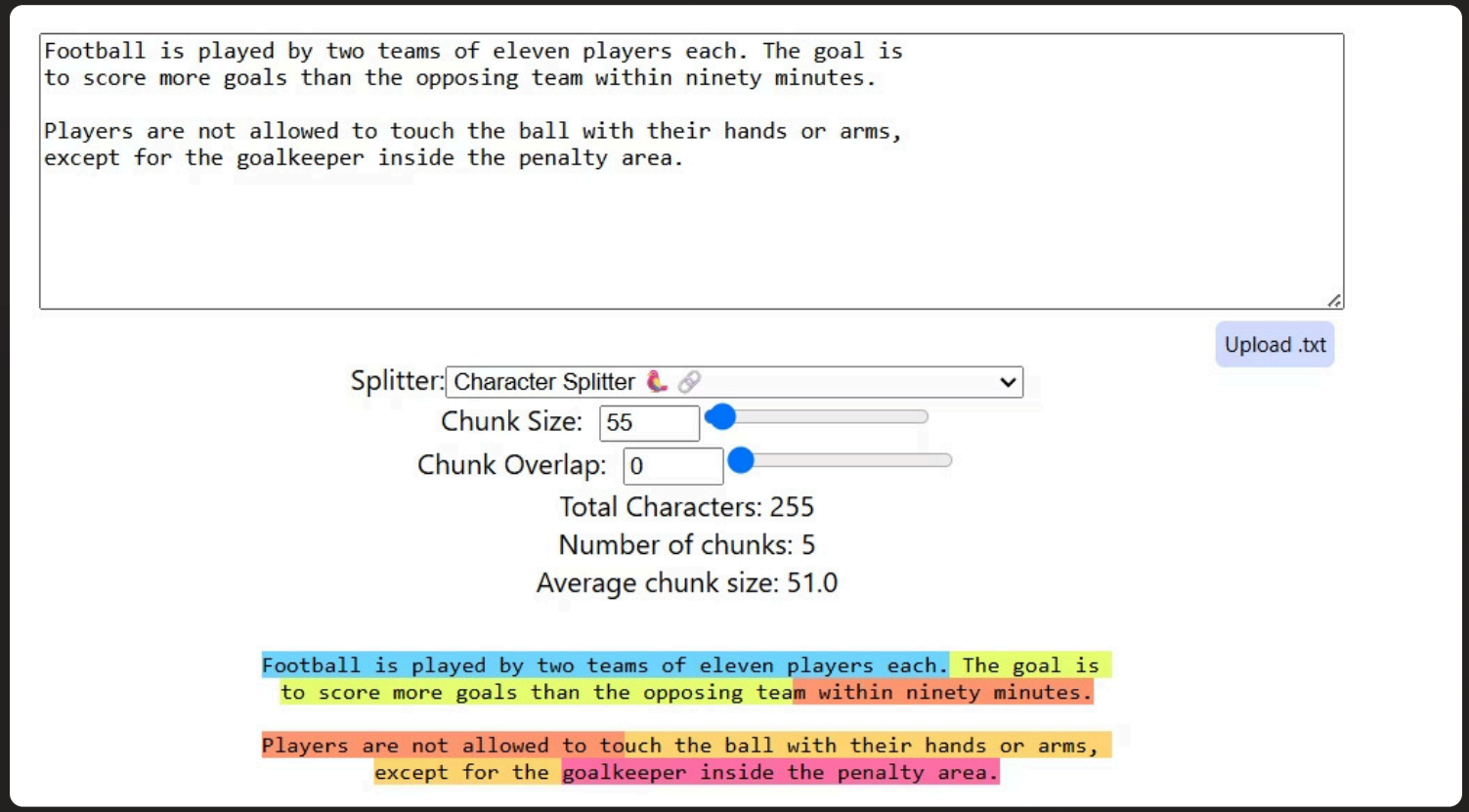
Size-based chunking is the simplest approach. Pick a number, split when you hit it, repeat. No structure awareness, no smart decisions, just counting.

How it works

Two main variants exist: character-based and token-based.

Character-based splitting is the simplest chunking method. It just counts letters (including spaces) and cuts the text once it reaches a set number.

For example, if `chunk_size = 55`, each chunk will have about 55 characters. A document with 255 characters will be cut into about 5 chunks.



problem: It doesn't care about words, sentences, or meaning — it only counts characters and cuts so the idea can get split in the middle and lose its meaning.

1. Size-based chunking

Token-based splitting

counts tokens instead of characters. This distinction matters because embedding models have token limits, not character limits. The word "unhappiness" is one word and 11 characters, but it might be 2 tokens ("un" + "happiness") depending on the tokenizer.

Like character-based splitting, it still cuts by count, not by meaning — so it can break sentences or ideas mid-way.

When to use this

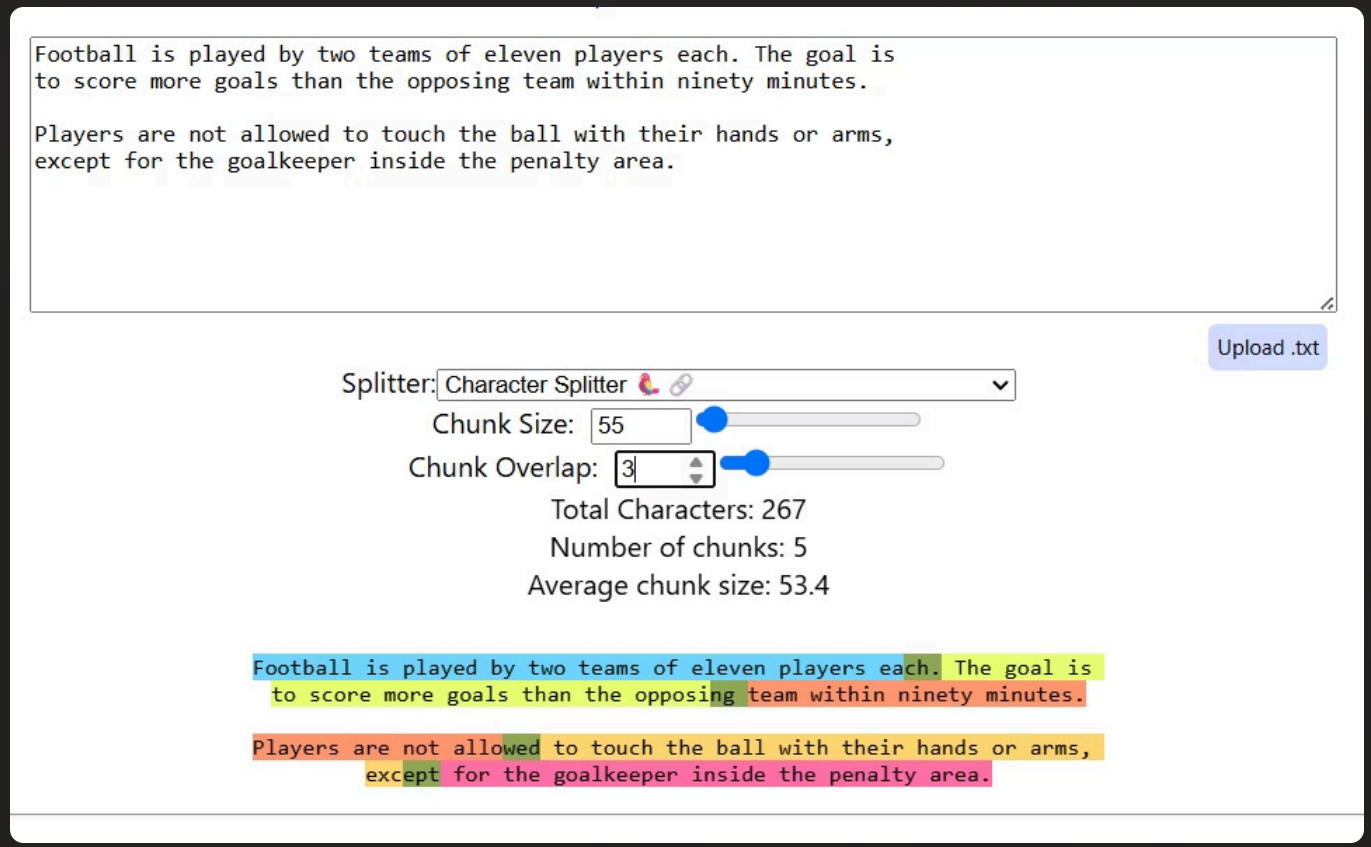
Size-based chunking works for:

- **Prototyping** : Get something working fast, optimize later
- **Uniform content**: News articles, product listings, short-form content with consistent structure
- **When simplicity matters**: Small teams, limited resources, straieghtforward use cases

If you're testing whether RAG works for your use case at all, start here. The implementation takes 5 minutes.

Chunk Overlap : Overlap means the end of one chunk repeats at the start of the next one.

Example: `chunk_size=55`, `overlap=3` → the last 3 characters of a chunk also appear at the start of the next chunk.**Why:** without overlap, each chunk starts completely fresh — no connection to what came before. Overlap keeps a small bridge, so meaning isn't lost right at the cut.

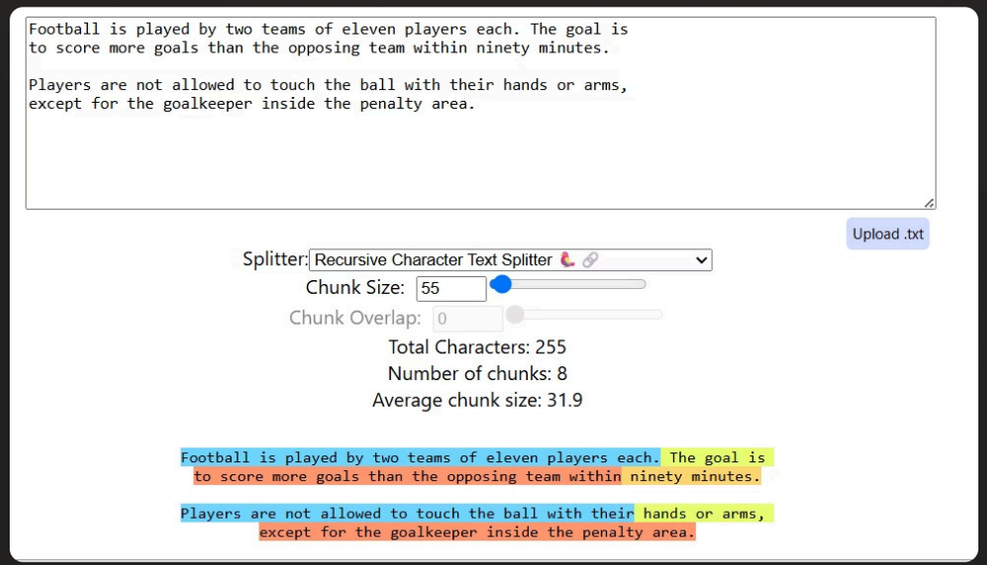


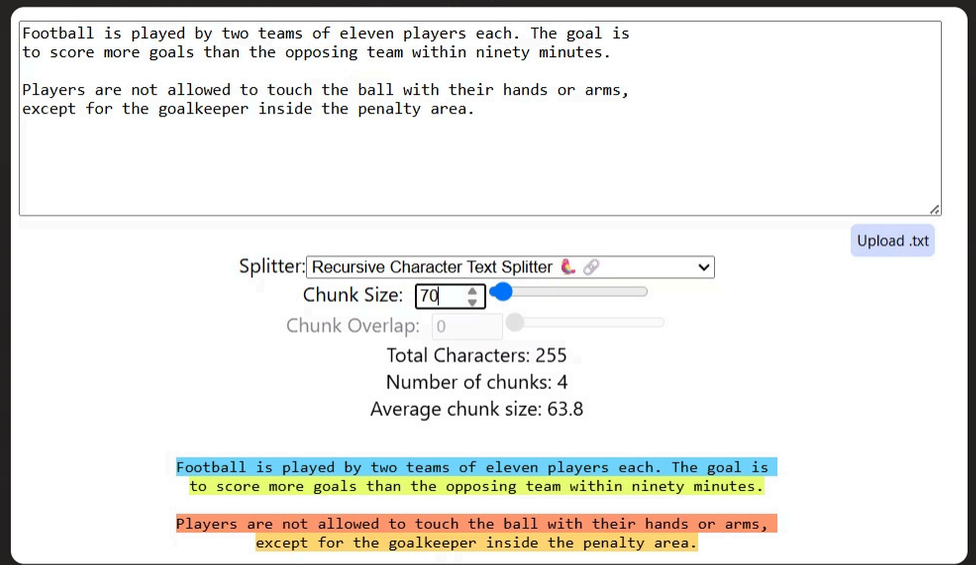
Trade-off: more overlap = more context, but more repeated text and slightly slower retrieval.

2. Recursive Chunking

This method is smarter. Instead of cutting blindly, respects natural language boundaries. It doesn't cut sentences mid-thought, and waits for paragraph breaks. If the paragraph is too long, it waits for a sentence. If the sentence is too long, it looks for a space between words.

Example :





2. Recursive Chunking

This method is smarter. Instead of cutting blindly, respects natural language boundaries. It doesn't cut sentences mid-

- Recursive splitting fixed: cutting words in half
- ❌ Recursive splitting did NOT fix: cutting one sentence/idea into two separate chunks(the problem)

The upside: chunks usually stay meaningful, because the split points make sense to a human reader too. This is why most RAG systems use this as their default method.

The downside: it's still based on size and structure, not real meaning — so two unrelated sentences that happen to be in the same paragraph could still end up in the same chunk.

Recursive splitter → overlap still helps, but less critical, since it already tries to cut at natural breaks first

3. Sentence-based chunking

Sentence-based chunking respects natural language boundaries. Instead of counting characters or tokens blindly, it identifies complete sentences and groups them into chunks.

How it works

The splitter first detects sentence boundaries using punctuation — **.** (period), **?** (question mark), **!** (exclamation point). It's smart enough to handle edge cases like "Dr. Smith" or "3.14" without treating them as sentence endings.

Once sentences are identified, the chunker groups them together to hit your target chunk size. It keeps adding sentences to the current chunk until the next sentence would push it over the limit — then it starts a new chunk.

Example (chunk_size = 100 tokens):

- Sentence 1 (40 tokens) + Sentence 2 (35 tokens) = 75 tokens → fits, add both to the same chunk
- Sentence 3 (45 tokens) → adding it would make 120 tokens, over the limit → start a new chunk instead

The result: chunks vary in size (one might be 90 tokens, another 77), but every sentence stays whole — never cut in the middle.

When to use this

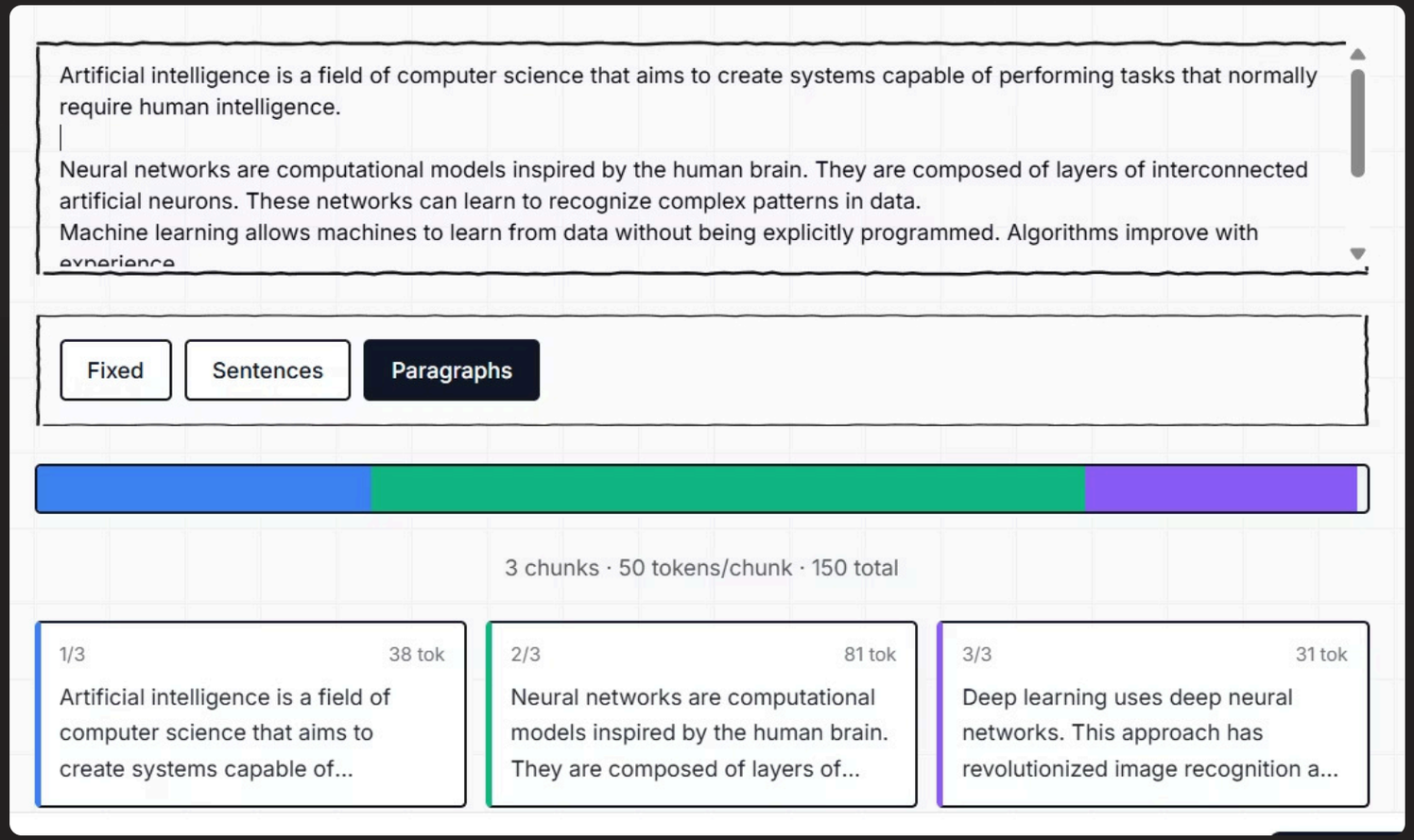
Sentence-based chunking works well for:

- Conversational data (chat logs, customer support transcripts)
- Q&A content where each question-answer pair should stay together
- Short-form content with clear sentence structure
- When preserving complete thoughts matters more than uniform chunk sizes

If your RAG system answers questions where context comes from complete sentences, this approach helps retrieval quality.

Paragraph-based Chunking

Definition: Paragraph-based chunking treats each paragraph as a complete unit and groups paragraphs together to build chunks — instead of counting characters or looking for sentence breaks.



Were it's used: Documents that are already organized into clear paragraphs — like articles, reports, or textbooks, where each paragraph usually represents one complete idea. This keeps related sentences together and preserves the author's natural structure, without needing an embedding model or LLM to decide boundaries.

Page-level Chunking

Definition:
Page-level chunking treats each page of a document as a separate chunk. Instead of counting tokens or looking for sentence breaks, it splits wherever the document's pagination naturally occurs.

How it works:
PDF files already have built-in page boundaries. Page-level chunking extracts content page by page and creates one chunk per page — or groups a few pages together if they're short.

Where it's used: Documents where layout matters — financial reports, research papers with figures, or medical records where each page holds a different type of information. Splitting across pages would lose that structure, so page-level chunking keeps each page whole.

4. Semantic Chunking
splitting by meaning, not size

How it is work:

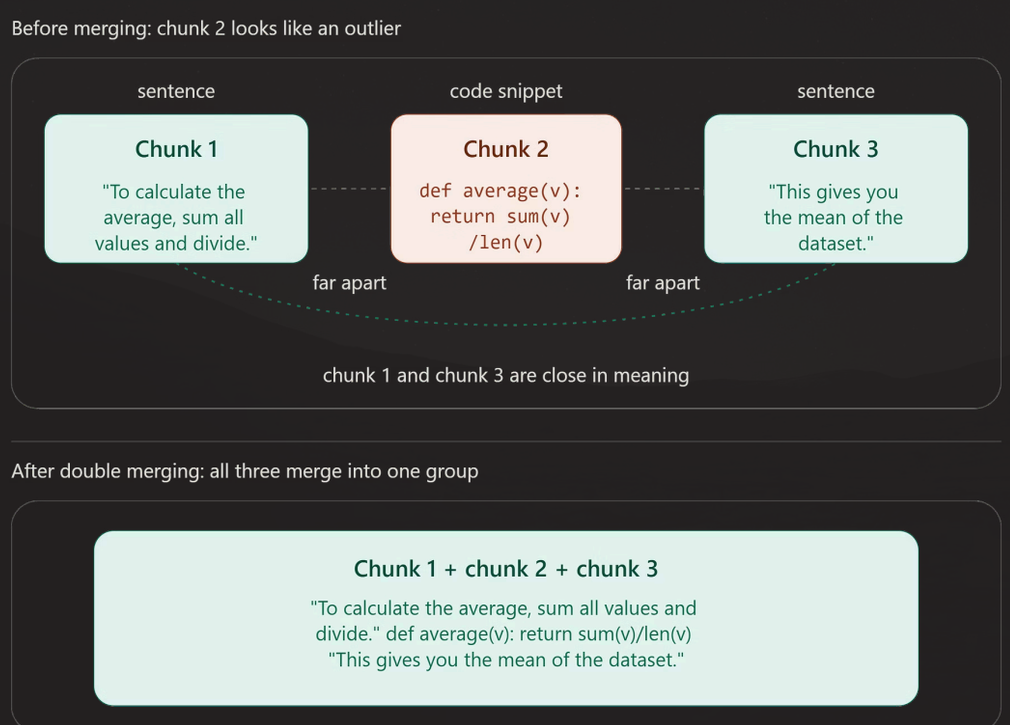
Step 1—Sentence segmentation: Break the document into individual sentences

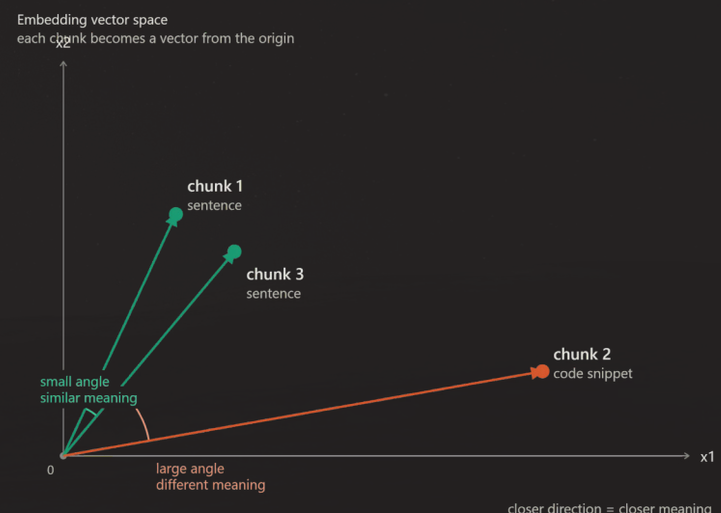
Step 2 — Embedding generation: Create vector embeddings for each sentence

Step 3 — Measure the distance between neighbors
The system compares each sentence to the one right after it, using cosine distance — a way to measure how different two vectors are. Same topic → small distance. Topic change → distance jumps.

Step 4 — Cut wherever the distance spikes
Wherever that distance crosses a threshold, a new chunk begins. Instead of counting characters, the system is watching for the moment the topic changes.

There's a refinement of this called semantic double merging. Sometimes a chunk in the middle — like a code snippet or formula — looks 'different' by embedding distance even though it logically belongs with the sentences around it. Double merging does a second pass: if the chunks before and after that odd one turn out to be very similar to each other, it merges all three back together instead of leaving that middle piece isolated.





5.Agentic Chunk

Definition:
Agentic chunking lets an AI agent decide on its own how to split a document into chunks — instead of following a fixed rule like character count or embedding distance.

How it works:
The document is first broken into small starting pieces, usually using recursive splitting. The AI agent then reads these pieces and decides how to group them into final chunks, based on topic and meaning. It can also add metadata — like a title or short summary — to each chunk, making it easier to find later during retrieval.

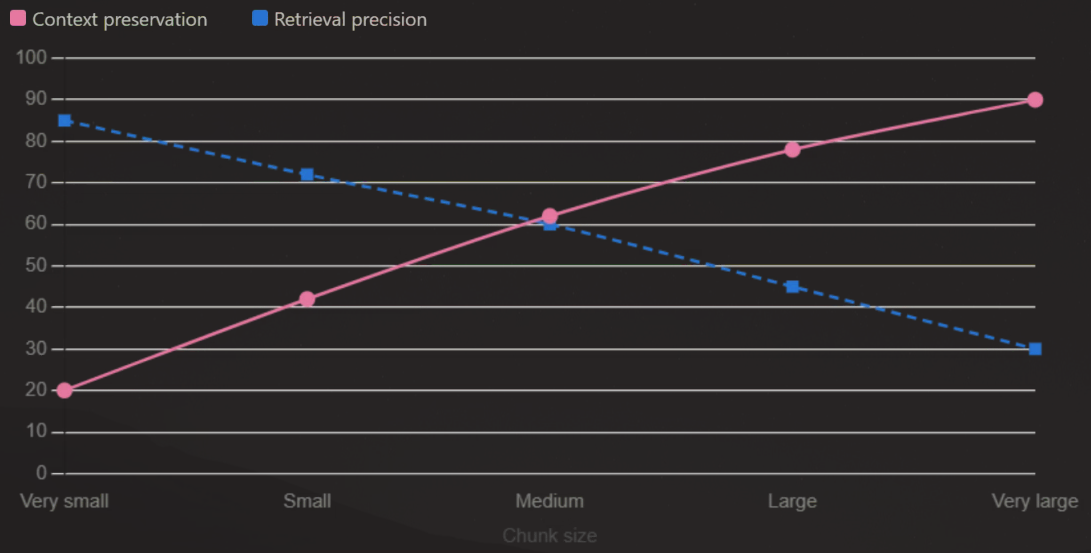
Where it's used:
Complex, high-value documents where getting the chunking exactly right matters more than the cost — for example, legal contracts, medical records, or highly technical documentation, where a wrong chunk boundary could cause a serious error.

Factors for choosing chunk size :

1. **Type of document** — technical text needs smaller chunks.
2. **Embedding model's limit** — the chunk must fit inside what the embedding model can handle (for example, 512 tokens).
3. **Type of question** — simple fact questions need small chunks. Complex questions need bigger chunks.
4. **LLM's limit** — how many chunks you send at once must fit inside the LLM's max input size.
5. **Document structure** — if the document already has clear parts (paragraphs, sections, Q&A), follow those instead of picking a random number.
6. **Testing** — in the end, you try different sizes and see which one gives the best answers. Testing beats guessing.

There's no fixed "best" size — it depends on the document, the model's limit, the question type, and testing.

The Effect of Chunk Size



The Core Trade-off

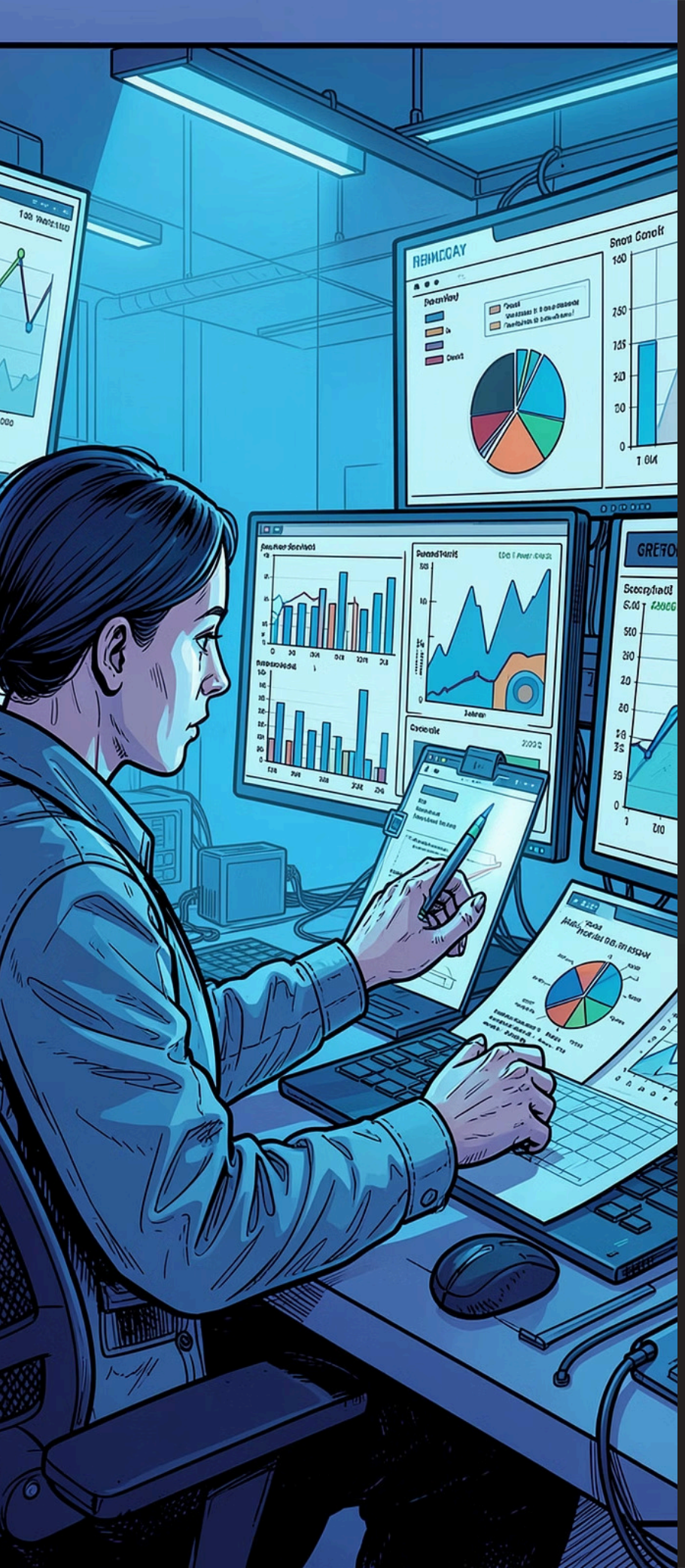
As chunk size increases, **retrieval precision decreases** while **context preservation increases.** The optimal chunk size balances these competing forces for your specific use case.

Small Chunks

High precision, low context — best for factual, pinpoint queries.

Large Chunks

High context, low precision — best for complex, multi-hop reasoning.



Key Research Insights

1

Chunk Size Isn't Everything

Embedding model choice and sensitivity significantly impact retrieval quality — sometimes more than chunk size alone.

2

Cross-File Context Matters

When answers span multiple documents, chunk boundaries and overlap strategy become critical for complete retrieval.

3

No Universal Best Size

Optimal chunk size depends on dataset domain, query complexity, and the embedding model in use.

4

Strategy Drives Quality

The chunking strategy directly determines RAG system quality — invest in thoughtful preprocessing design.

THANK YOU